

Sesión 1: Introducción e Instalación del Ecosistema Flutter/Android

Introducción

Bienvenido a la primera sesión del curso de **Desarrollo de Aplicaciones Móviles Nativas con Flutter**. En esta sesión sentaremos las bases conceptuales, técnicas y prácticas sobre las cuales construirás software móvil de nivel profesional a lo largo del semestre.

Durante años, la industria del desarrollo móvil estuvo dividida en dos grandes paradigmas: el **desarrollo nativo puro** (usando Kotlin/Java para Android y Swift/Objective-C para iOS) y el **desarrollo híbrido o basado en WebViews** (como PhoneGap, Cordova o Ionic). El primero ofrecía un rendimiento excepcional y acceso directo al hardware del dispositivo, pero exigía duplicar esfuerzos de desarrollo, mantener dos códigos fuente distintos y contratar equipos especializados para cada plataforma. El segundo prometía "escribir una vez y ejecutar en todos lados", pero pagaba un costo altísimo en rendimiento, ofreciendo interfaces lentas, poco fluidas y una experiencia de usuario distante de los estándares visuales de cada sistema operativo.

Posteriormente surgieron frameworks reactivos de puente (*bridge*), como React Native, que traducían componentes de JavaScript a widgets nativos en tiempo de ejecución. Si bien esto mejoró sustancialmente la experiencia, la presencia del puente de comunicación continuaba siendo un cuello de botella para animaciones complejas, cálculo intensivo de datos y renderizado a alta velocidad.

Aquí es donde entra **Flutter**. Desarrollado por Google y presentado oficialmente en 2017, Flutter rompió con estos esquemas tradicionales al introducir un enfoque revolucionario: **un motor de renderizado propio en C++ que dibuja pixel por pixel la interfaz de usuario**, prescindiendo completamente de los componentes nativos de la plataforma y de los puentes de traducción en tiempo de ejecución.

En esta sesión exploraremos qué es Flutter, por qué utiliza el lenguaje **Dart**, cuál es la arquitectura que le permite alcanzar rendimientos de 60 a 120 cuadros por segundo (FPS), y realizaremos el proceso completo de instalación y configuración del SDK, Android Studio, el juego de herramientas de Android (*Android Toolchain*) y la verificación del entorno mediante **flutter doctor**.

Conceptos y Definiciones

Para comprender el funcionamiento de Flutter desde una perspectiva de ingeniería de software, es imprescindible dominar los siguientes conceptos clave:

1. ¿Qué es Flutter?

Flutter no es únicamente una biblioteca de interfaz de usuario; es un **Software Development Kit (SDK)** multiplataforma de código abierto. Permite compilar aplicaciones nativas de alto rendimiento para Android, iOS, Web, Windows, macOS y Linux a partir de una única base de código (*single codebase*).

A diferencia de otros frameworks, Flutter no convierte su código a vistas nativas del sistema operativo (**`android.widget.Button`** o **`UIButton`**). En su lugar, Flutter funciona de forma análoga a un motor de

videojuegos (como Unity o Unreal Engine): toma el control de una superficie de lienzo en la pantalla (*Canvas*) y dibuja cada botón, texto, animación y tarjeta utilizando su propio motor gráfico.

2. El Lenguaje Dart

Dart es el lenguaje de programación orientado a objetos, fuertemente tipado y de código abierto desarrollado por Google que impulsa a Flutter. Diseñado originalmente para el desarrollo web estructurado, Dart posee características ideales para la creación de interfaces de usuario modernas:

- **Sound Null Safety (Seguridad nula sólida):** Previene en tiempo de compilación los errores de referencia nula (el famoso *NullPointerException*), garantizando que los valores no sean nulos a menos que se declaren explícitamente como opcionales (*Type?*).
- **Compilación Dual (JIT y AOT):**
- **JIT (Just-In-Time):** Compila el código al vuelo durante la fase de desarrollo. Esto habilita la funcionalidad **Stateful Hot Reload**, permitiendo ver los cambios en el código en subsegundos sin perder el estado actual de la aplicación.
- **AOT (Ahead-Of-Time):** Compila el código Dart directamente a código de máquina nativo del procesador (ARM64 o x86_64) para la versión de producción. Esto elimina intermediarios y garantiza arranques instantáneos y máxima velocidad de ejecución.
- **Recolección de Basura Generacional (*Generational Garbage Collection*):** Optimizado para crear y destruir miles de objetos de interfaz de usuario de corta vida por segundo sin generar pausas en el hilo de renderizado.

3. La Arquitectura de Tres Capas de Flutter

El SDK de Flutter se organiza estructuralmente en tres capas superpuestas, cada una con responsabilidades bien delimitadas:

1. **La Capa Superior (Framework en Dart):** Es la capa con la que interactuamos directamente como desarrolladores. Contiene las bibliotecas de alto nivel:
 - **Material Design y Cupertino:** Colecciones de widgets que implementan los lenguajes de diseño visual de Google y Apple.
 - **Widgets Core:** Clases base para disposición visual, texto, gestión de gestos y animaciones.
 - **Rendering, Animation, Painting:** Capas de abstracción para cálculo de restricciones geométricas y pintado.
2. **La Capa Intermedia (Engine en C/C++):** Es el corazón del framework. Contiene el motor gráfico **Impeller** (o Skia en versiones legadas), la máquina virtual de Dart (Dart VM), el subsistema de renderizado de texto, y las bibliotecas para la gestión de hilos de ejecución (*Isolates*).
3. **La Capa Inferior (Embedder Nativo):** Es un componente específico para cada plataforma (escrito en Java/Kotlin para Android, Objective-C/Swift para iOS). Se encarga de coordinar el acceso al lienzo de la pantalla nativa, gestionar los eventos de entrada (toques, teclado), la energía, y la comunicación con servicios nativos mediante canales de plataforma (*Platform Channels*).

```

+-----+
|          FRAMEWORK (Escrito en Dart)          |
| [Material / Cupertino] -> [Widgets] -> [Rendering] -> [Animation] |
+-----+
|          ENGINE (Escrito en C/C++)            |
| [Impeller/Skia Engine] | [Dart Runtime VM] | [Text Layout] |
+-----+
|          EMBEDDER (Nativo)                    |
| [Android (Kotlin/Java)] | [iOS (Swift/Obj-C)] | [Surface] |
+-----+

```

4. La Cadena de Herramientas de Android (Android Toolchain)

Para que Flutter pueda generar un ejecutable **.apk** o **.aab** (Android App Bundle), requiere apoyarse en el juego de herramientas oficial de Android. Esto incluye:

- **Android SDK Build-Tools:** Compiladores y empaquetadores como **d8** (para convertir bytecode de Java a DEX) y **aapt2**.
- **Gradle:** El sistema de automatización de compilación (*build system*) utilizado por Android para gestionar dependencias, variantes de compilación y empaquetado final.
- **Android NDK (Native Development Kit):** Necesario cuando se requiere compilar código nativo de C/C++ dentro de proyectos Android.
- **Licencias del SDK:** Acuerdos legales de uso del SDK de Android que deben ser aceptados explícitamente por el desarrollador en la terminal antes de compilar.

Guía de Instalación y Configuración del Entorno (Paso a Paso)

A continuación, realizaremos la instalación paso a paso en un sistema operativo basado en **Linux/Windows** orientado a entornos de desarrollo para Android.

Paso 1: Requisitos de Sistema

Asegúrate de contar con un equipo que cumpla con los siguientes mínimos:

- **Sistema Operativo:** Windows 10/11 (64 bits) o distribución Linux (Ubuntu 22.04 LTS o superior).
- **Procesador:** Intel Core i5/i7 o AMD Ryzen 5/7 de 4ª generación en adelante.
- **Memoria RAM:** Mínimo 8 GB (Recomendado: 16 GB o más si ejecutarás el emulador Android).
- **Almacenamiento:** Mínimo 20 GB de espacio libre en disco de estado sólido (SSD).
- **Virtualización Habilitada:** Habilitar **Intel VT-x / AMD-V** en la BIOS/UEFI de la computadora para ejecutar emuladores Android con aceleración de hardware (HAXM o KVM).

Paso 2: Descarga e Instalación del SDK de Flutter

1. Dirígete al sitio web oficial de Flutter: [<https://docs.flutter.dev/get-started/install>] (<https://docs.flutter.dev/get-started/install>).
2. Descarga el paquete estable (*Stable Channel*) correspondiente a tu sistema operativo.

3. Extrae el archivo comprimido en una ruta de directorio dedicada que no requiera permisos especiales de administrador.

- **Linux:** `~/development/flutter`
- **Windows:** `C:\src\flutter` (Evita instalar Flutter en rutas con espacios o caracteres especiales como `C:\Program Files\`).

Paso 3: Configuración de Variables de Entorno (PATH)

Es imprescindible agregar el directorio ejecutable de Flutter (`flutter/bin`) a tu variable de entorno global `PATH` para poder ejecutar comandos desde cualquier terminal.

En Linux / macOS (Bash o Zsh):

Abre tu archivo de configuración de terminal (`~/.bashrc` o `~/.zshrc`) y agrega al final la siguiente línea:

```
export PATH="$HOME/development/flutter/bin:$PATH"
```

Aplica los cambios en la terminal actual:

```
source ~/.bashrc
```

En Windows:

1. Abre el menú Inicio, busca "**Editar las variables de entorno del sistema**" y presiona Enter.
2. Haz clic en el botón **Variables de entorno...**
3. En la sección **Variables de usuario**, selecciona la variable `Path` y haz clic en **Editar...**
4. Haz clic en **Nuevo** y agrega la ruta completa a la carpeta `bin` de Flutter (ej. `C:\src\flutter\bin`).
5. Acepta y cierra todas las ventanas.

Paso 4: Instalación de Android Studio y SDK Tools

1. Descarga e instala **Android Studio** desde [<https://developer.android.com/studio>] (<https://developer.android.com/studio>).
2. Ejecuta el asistente de configuración inicial (*Setup Wizard*) de Android Studio e instala:
 - **Android SDK Platform** (Versión más reciente disponible, ej. API 34).
 - **Android SDK Build-Tools**.
 - **Android Emulator**.
3. Una vez abierto Android Studio, ve a **More Actions > SDK Manager** (o *Tools > SDK Manager*).
4. Selecciona la pestaña **SDK Tools**.
5. Marca la casilla **Android SDK Command-line Tools (latest)** y presiona **Apply** para instalarlas. *Este paso es vital; sin estas herramientas, Flutter no podrá comunicarse con el SDK de Android.*

```
[SDK Manager Window]
├─ SDK Platforms
├─ SDK Tools
│   ├── Android SDK Build-Tools 34
│   ├── Android SDK Command-line Tools (latest) <-- [OBLIGATORIO]
│   ├── Android Emulator
│   └─ Android SDK Platform-Tools
```

Paso 5: Aceptación de Licencias de Android

Abre una terminal limpia de tu sistema operativo y ejecuta el siguiente comando para revisar y aceptar las licencias de uso de Android:

```
flutter doctor --android-licenses
```

El sistema te presentará una serie de acuerdos de licencia. Presiona la tecla **y** y luego **Enter** en cada prompt para aceptarlas todas.

Paso 6: Verificación del Entorno con Flutter Doctor

Ejecuta el comando de diagnóstico oficial de Flutter:

```
flutter doctor
```

El comando escaneará tu sistema operativo en busca de dependencias faltantes. Si todo está configurado correctamente, verás una salida con marcas de verificación verdes similar a la siguiente:

```
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.22.0, on Linux, locale es_MX.UTF-8)
[✓] Android toolchain - develop for Android devices (Android SDK version 34.0.0)
[✓] Chrome - develop for the web
[✓] Android Studio (version 2023.3)
[✓] VS Code (version 1.89.0)
[✓] Connected device (2 available)
[✓] Network resources

• No issues found!
```

Si aparece algún símbolo **[!]** o **[X]**, el mismo informe de **flutter doctor** te indicará la instrucción exacta que debes ejecutar para solucionar el problema.

Ejemplo Completo con Código Fuente: Hola Mundo en Flutter

A continuación, crearemos, analizaremos y ejecutaremos nuestra primera aplicación.

1. Creación del Proyecto desde la Terminal

Abre tu terminal, navega a la carpeta donde guardas tus proyectos de desarrollo y ejecuta:

```
flutter create hola_mundo_escom
```

Este comando creará una estructura completa de directorios que contiene archivos para Android, iOS, Web y escritorio, junto con la carpeta `lib/` donde residirá nuestro código en Dart.

Navega al directorio recién creado:

```
cd hola_mundo_escom
```

2. Inspección del Código Fuente

Abre el archivo `lib/main.dart` en tu editor de código preferido (VS Code o Android Studio). Reemplaza todo su contenido por el siguiente código estructurado de manera explícita y profesional:

```
import 'package:flutter/material.dart';

/// Punto de entrada principal de la aplicación.
/// La función [main] es la primera en ejecutarse en el Isolate de Dart.
void main() {
  // Invoca la función runApp() pasando la raíz de nuestra jerarquía de Widgets.
  runApp(const MiAplicacionHolaMundo());
}

/// Widget raíz de la aplicación.
/// Hereda de [StatelessWidget] ya que su estructura no cambia dinámicamente
/// durante su ciclo de vida inicial.
class MiAplicacionHolaMundo extends StatelessWidget {
  // Constructor constante para optimización de memoria.
  const MiAplicacionHolaMundo({super.key});

  @override
  Widget build(BuildContext context) {
    // MaterialApp configura el tema, la navegación y las herramientas de Material
    Design.
    return MaterialApp(
      title: 'Hola Mundo ESCOM',
      debugShowCheckedModeBanner: false, // Remueve el banner de depuración
```

```

    theme: ThemeData(
      useMaterial3: true,
      colorScheme: ColorScheme.fromSeed(
        seedColor: const Color(0xFF800020), // Color Guinda institucional
      ),
    ),
    home: const PantallaPrincipal(),
  );
}
}

/// Pantalla principal de la aplicación.
class PantallaPrincipal extends StatelessWidget {
  const PantallaPrincipal({super.key});

  @override
  Widget build(BuildContext context) {
    // Scaffold provee la estructura visual básica de una pantalla Material
    (AppBar, Body, etc.)
    return Scaffold(
      appBar: AppBar(
        title: const Text('ESCOM - IPN Flutter App'),
        centerTitle: true,
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
      ),
      body: Center(
        // Centra el contenido en el eje horizontal y vertical
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            const Icon(
              Icons.developer_mode,
              size: 80.0,
              color: Color(0xFF800020),
            ),
            const SizedBox(height: 20.0), // Espaciador vertical
            Text(
              '¡Hola Mundo desde la ESCOM!',
              style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                fontWeight: FontWeight.bold,
              ),
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 10.0),
            const Text(
              'Ingeniería en Sistemas Computacionales',
              style: TextStyle(
                fontSize: 16.0,
                color: Colors.grey,
                fontStyle: FontStyle.italic,
              ),
            ),
          ],
        ),
      ),
    ),
  ),
}

```

```
    ),  
  );  
}  
}
```

3. Ejecución de la Aplicación

1. Conecta tu dispositivo móvil Android mediante un cable USB con la **Depuración USB** activada en las Opciones de Desarrollador, o bien inicia un Emulador Android desde el **Device Manager** de Android Studio.
2. En la terminal dentro de la carpeta del proyecto, verifica que Flutter detecte el dispositivo:

```
flutter devices
```

3. Ejecuta la aplicación en el dispositivo detectado:

```
flutter run
```

Flutter procederá a compilar el proyecto mediante Gradle, instalar el ejecutable APK en tu dispositivo y establecer la conexión de depuración.

Prácticas Guiadas

Práctica 1: Experimentación con Hot Reload

1. Con la aplicación ejecutándose en tu emulador o dispositivo real mediante el comando `flutter run`.
2. Dirígete al archivo `lib/main.dart` en tu editor.
3. Modifica la cadena de texto `'¡Hola Mundo desde la ESCOM!'` por `'¡Bienvenido al Desarrollo Móvil con Flutter!'`.
4. Modifica el tamaño del icono de `80.0` a `120.0`.
5. Guarda el archivo (`Ctrl + S` en Windows/Linux o `Cmd + S` en macOS) o presiona la tecla `r` en la terminal donde ejecutas `flutter run`.
6. Observa cómo la pantalla de tu dispositivo se actualiza de manera casi instantánea (menos de 500 milisegundos) conservando el estado de la aplicación.

Práctica 2: Análisis de la Estructura de Proyectos

Abre el explorador de archivos de tu editor y examina la anatomía del directorio creado por Flutter:

- `android/`: Contiene el proyecto nativo de Android manejado por Gradle. Aquí se configuran los permisos en el archivo `AndroidManifest.xml` y la versión mínima del SDK (`build.gradle`).
- `ios/`: Proyecto nativo para iOS manejado por Xcode (requiere macOS para compilar).

- **lib/**: Es el directorio de trabajo principal. Contiene todo el código Dart de la aplicación.
 - **pubspec.yaml**: El archivo de configuración más importante del proyecto. Define el nombre de la aplicación, versión, dependencias externas (paquetes de **pub.dev**), fuentes tipográficas y recursos estáticos (imágenes, audios).
 - **.gitignore**: Archivo que especifica qué archivos temporales o compilados deben ser ignorados por el sistema de control de versiones Git.
-

Resumen y Puntos Clave

En esta primera sesión hemos sentado las bases indispensables para trabajar con el framework Flutter. A continuación, se presentan los conceptos clave que debes recordar:

- **Paradigma de Renderizado**: Flutter no utiliza componentes visuales nativos del sistema ni puentes de traducción JavaScript en tiempo de ejecución. Dibuja su propia interfaz sobre un **Canvas** usando su motor gráfico de alto rendimiento escrito en C++ (**Impeller/Skia**).
 - **Lenguaje Dart**: Utiliza Dart debido a su tipado fuerte, seguridad nula sólida (*Sound Null Safety*), recolección de basura eficiente para UI y compilación dual: **JIT** para desarrollo ágil (*Hot Reload*) y **AOT** para producción nativa de alto rendimiento.
 - **Arquitectura de 3 Capas**: El SDK se divide en el **Framework** (Dart), el **Engine** (C/C++) y el **Embedder** (Nativo para cada SO).
 - **Android Toolchain**: Para compilar en Android es obligatorio contar con el SDK de Android, las **Android SDK Command-line Tools**, Gradle y haber aceptado todas las licencias mediante **flutter doctor --android-licenses**.
 - **Flutter Doctor**: Es la herramienta de diagnóstico integrada principal para verificar que el entorno de desarrollo no tenga dependencias rotas o desconfiguradas.
-

Bibliografía y Referencias

1. **Google LLC**. (2024). *Flutter Documentation: Technical Overview*. Recuperado de: <https://docs.flutter.dev/resources/architectural-overview>
2. **Dart Language Team**. (2024). *Dart Language Guide: Type System and Null Safety*. Recuperado de: <https://dart.dev/guides/language/evolution>
3. **Windmill, E.** (2020). *Flutter in Action*. Manning Publications. ISBN: 978-1617296147.
4. **Napoli, M.** (2022). *Beginning Flutter: A Hands-On Guide to App Development*. Wrox. ISBN: 978-1119550822.
5. **Android Developers**. (2024). *Command line tools release notes*. Recuperado de: <https://developer.android.com/studio/releases/cmdline-tools>